

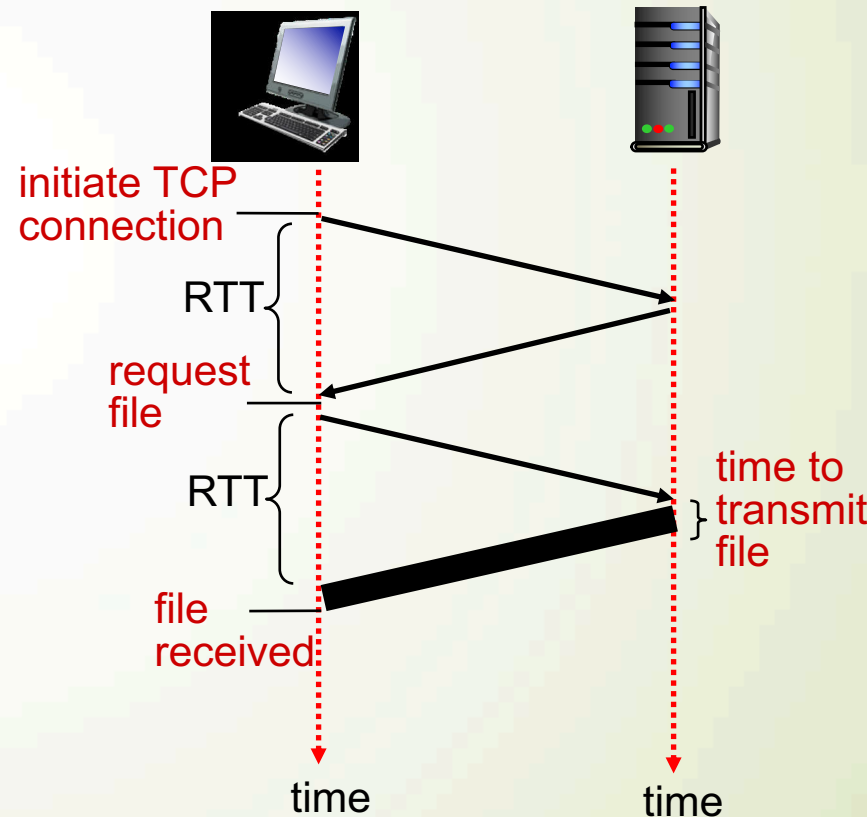
Response Time of Non-Persistent HTTP

One rrt to initiate tcp connection

One rrt for http request and first few bytes of http response to return

File transmission time

- Sum of:
 - one RTT to initiate TCP connection
 - one RTT for HTTP request and first few bytes of HTTP response to return
 - file transmission time
- Hence, non-persistent HTTP response time = $2RTT + \text{file transmission time}$



Persistent HTTP

Server leaves connection open after sending response

Subsequent http messages between same client server sent over open connection

- server leaves connection open after sending response
- subsequent HTTP messages between same client/server sent over open connection
 - Client sends requests as soon as it encounters a referenced object
- client sends requests as soon as it encounters a referenced object
 - As little as one rtt for all the referenced objects
- as little as one RTT for all the referenced objects
 - Shorter response time lower os and network overhead
- advantages: shorter response time, lower OS and network overhead

User-Server State: Cookies

four components:

Cookie header line
of http response
message

cookie header
line of HTTP
response
message

Next http
request message

cookie header
line in next HTTP
request message

Cookie file kept on
user's host managed
by user's browser

cookie file kept on
user's host,
managed by
user's browser

Back-end database
at website

back-end
database at Web
site

Susan visits
specific e-
commerce site
for first time

when initial HTTP
requests arrives at
site, site creates:

unique ID

entry in backend
database for ID

one week later the
same cookie is sent
back to the same
website

What Cookies Can Do?

- Keep user states for:
 - authorization
 - shopping carts
 - recommendations
 - user session state (Web e-mail)
- **Q: Didn't we say HTTP is stateless?**
- privacy concern
 - cookies permit sites to learn a lot about you
 - you may supply name and e-mail to sites

Agenda

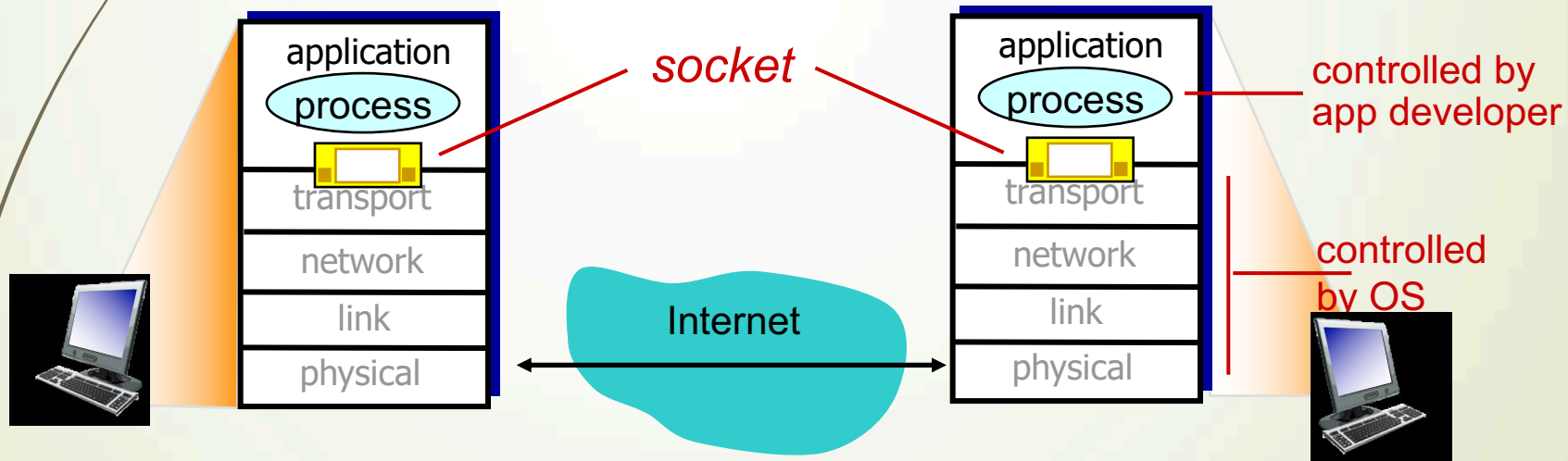
- Architecture: Client-Server versus P2P
- Application Service Requirements: TCP versus UDP
- Sample Application Protocol: HTTP
- Socket Programming in Python

How Processes Communicate?

- **process**: program running within a host
- within same host, two processes communicate using inter-process communication (**IPC**, defined by OS)
- processes in different hosts communicate by exchanging **messages**
- **client process**: process that **initiates** communication
- **server process**: process that **waits** to be contacted
- Note: Both client-server and P2P applications have client **and** server process

Sockets

- process sends/receives messages to/from its **sockets**



Addressing...

- ▶ to receive messages, process must have identifier
- ▶ host device has unique 32-bit IP address
- ▶ but, many processes can be running on same host
- ▶ identifier includes both IP address and port numbers
- ▶ example port numbers:
 - ▶ HTTP server: 80
 - ▶ mail server: 25
- ▶ to send HTTP message to `gaia.cs.umass.edu` web server:
 - ▶ IP address: 128.119.245.12
 - ▶ port number: 80

IP routers route each packet using the 5-tuple in the header `<src_ip, src_port, dst_ip, dst_port, proto>`

Socket Programming with UDP

- UDP: **no “connection”** between client & server
 - no handshaking before sending data
 - sender explicitly attaches IP destination address and port # to each packet
 - receiver extracts sender IP address and port# from received packet
- UDP: transmitted data may be **lost** or received **out-of-order**
- Application viewpoint:
 - UDP provides unreliable transfer of **groups of bytes** (“datagrams”) between client and server ← nature boundary between adjacent messages

Client/Server Socket Interaction: UDP

server (running on serverIP)

create socket, port= x:

```
serverSocket =  
socket(AF_INET,SOCK_DGRAM)
```

↓
read datagram from
serverSocket

↓
write reply to
serverSocket
specifying
client address,
port number

client

create socket:

```
clientSocket =  
socket(AF_INET,SOCK_DGRAM)
```

↓
Create datagram with server IP and
port=x; send datagram via
clientSocket

↓
read datagram from
clientSocket

↓
close
clientSocket



UDP Client Example

Python UDPClient

include Python's socket library

→ `from socket import *`

`serverName = 'hostname'`

`serverPort = 12000`

create UDP socket for server

→ `clientSocket = socket(AF_INET, SOCK_DGRAM)`

get user keyboard input

→ `message = raw_input('Input lowercase sentence:')`

Attach server name, port to message; send into socket

→ `clientSocket.sendto(message.encode(), (serverName, serverPort))`

read reply characters from socket into string

→ `modifiedMessage, serverAddress = clientSocket.recvfrom(2048)`

print out received string and close socket

→ `print modifiedMessage.decode()`
`clientSocket.close()`

UDP Server Example

Python UDPServer

```
from socket import *
serverPort = 12000

create UDP socket → serverSocket = socket(AF_INET, SOCK_DGRAM)
bind socket to local port → serverSocket.bind(("", serverPort))
number 12000

print ("The server is ready to receive")

loop forever → while True:
    Read from UDP socket into → message, clientAddress = serverSocket.recvfrom(2048)
    message, getting client's → modifiedMessage = message.decode().upper()
    address (client IP and port)

    send upper case string → serverSocket.sendto(modifiedMessage.encode(),
    back to this client → clientAddress)
```

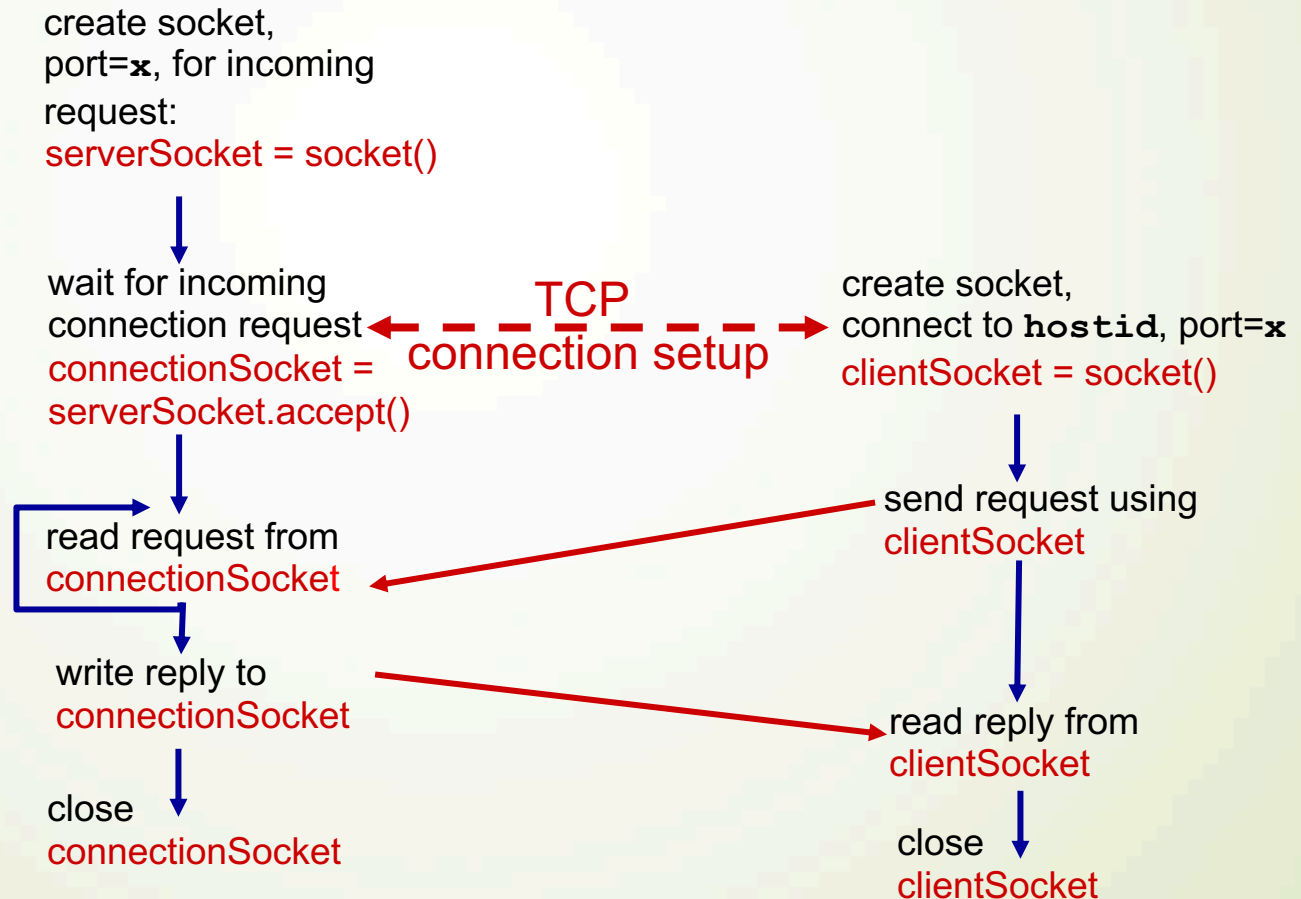
Socket Programming with TCP

- **client must contact server**
 - server process must first be running
 - server must have created socket (door) that welcomes client's contact
- **client contacts server by:**
 - Creating TCP socket, specifying IP address, port number of server process
 - when client creates socket: client TCP establishes connection to server TCP
- when contacted by client, server TCP creates **new socket** for server process to communicate with that particular client
 - allows server to talk with **multiple** clients
 - source port numbers used to distinguish clients (5-tuple)
- application viewport:
 - TCP provides reliable, **in-order byte-stream transfer** ("pipe") between client and server ← no nature boundary between two messages

Client/Server Socket Interaction: TCP

server (running on `hostid`)

client



TCP Client Example

Python TCPClient

create TCP socket for
server, remote port 12000

No need to attach server
name, port

```
from socket import *
serverName = 'servername'
serverPort = 12000
clientSocket = socket(AF_INET, SOCK_STREAM)
clientSocket.connect((serverName, serverPort))
sentence = raw_input('Input lowercase sentence:')
clientSocket.send(sentence.encode())
modifiedSentence = clientSocket.recv(1024)
print ('From Server:', modifiedSentence.decode())
clientSocket.close()
```

TCP Server Example

Python TCPServer

```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET, SOCK_STREAM)
serverSocket.bind(('', serverPort))
serverSocket.listen(1)
print 'The server is ready to receive'
while True:
    connectionSocket, addr = serverSocket.accept()
    sentence = connectionSocket.recv(1024).decode()
    capitalizedSentence = sentence.upper()
    connectionSocket.send(capitalizedSentence.
                           encode())
    connectionSocket.close()
```

create TCP server socket →

server begins listening for incoming TCP requests →

loop forever →

server waits on accept() for incoming requests, new socket created on return →

read bytes from socket (but not address as in UDP) →

close connection to this client (but *not* server socket) →

Take-Away Messages

- ▶ typical request/reply message exchange:
 - ▶ client requests info or service
 - ▶ server responds with data, status code
- ▶ message formats:
 - ▶ **headers**: fields giving info about data
 - ▶ **payload**: info (**data**) being communicated
- ▶ important design decisions
 - ▶ control vs. messages
 - ▶ **in-band, out-of-band**
 - ▶ centralized vs. decentralized
 - ▶ stateless vs. stateful
 - ▶ reliable vs. unreliable message transfer
- ▶ philosophy: **complexity at network edge**

Questions

chsu@cs.nthu.edu.tw

